

SIT225 DATA CAPTURE TECHNOLOGY

TASK 5C

NAME: Srishti

STUDENT ID: 225617009

DATE: 12th September 2026

AI Statement:

I confirm that I have not used any Generative Artificial Intelligence (GenAI) tools to produce, write, or complete the work submitted for this assessment. The work is my own and reflects my understanding of the subject matter.

1) Method for Smoothly Plotly Dash Updates

The main problem is that the accelerometer samples arrive from the phone, via Arduino IoT Cloud, at an irregular rate – sometimes a few values close together, sometimes a gap. If the Dash graph was directly redrawn inside the cloud callback (on_x/on_y/on_z), then the redraw rate would inherit that irregularity and user would see stuttering or visible 'jump cuts' between frames instead of a continuous line.

Here we present an approach that decouples data arrival and data display by two independent loops that communicate only through a shared thread-safe rolling buffer:

1. Data arrival (producer)

Every time a new value for that particular property is pushed from the phone, Arduino IoT Cloud calls on_x, on_y and on_z. Because the three axes are written by different callbacks, save_combined_data() only commits a row (to CSV) and to the plotting buffer when all three flags x_received, y_received and z_received are True for the current cycle, then resets the flags. This keeps the three series in sync, rather than each axis drifting in at its own rate.

2. Rolling buffer (shared state)

add_sample() pushes the new tuple (t,x,y,z) into 4 collections.deque objects, each with a maxlen of 200, and . . . By using a deque with a maxlen, the oldest element is automatically dropped when the buffer is full, so this acts like a fixed-size sliding window over the last ~200 samples without requiring any manual trimming logic. This buffer is the only common resource between the cloud thread and the Dash thread.

3. Display (consumer)

The cloud data actually arrived when it did, but a `dcc.Interval` component calls `update_graph()` on a fixed 50 ms clock. For each tick, the callback just reads the current contents of the deques (`list(time_buffer)`, `list(x_buffer)`...) and rebuilds the Plotly figure. Since the read is decoupled from the write, the graph redraws at a constant cadence even if the underlying sensor stream is bursty — gaps are simply redrawn with the last known buffer state instead of freezing or jumping.

4. Concurrency

`client.start()` (the Arduino Cloud event loop) is blocking so it is run on a background daemon thread (`threading.Thread(target=start_cloud, daemon=True)`).while `app.run()` (the Dash/Flask server) owns the main thread. `start()` So the cloud connection cannot block the web server and vice-versa.

5. Visual continuity settings

There are two Plotly/Dash settings that are designed for perceived smoothness, not raw data handling:

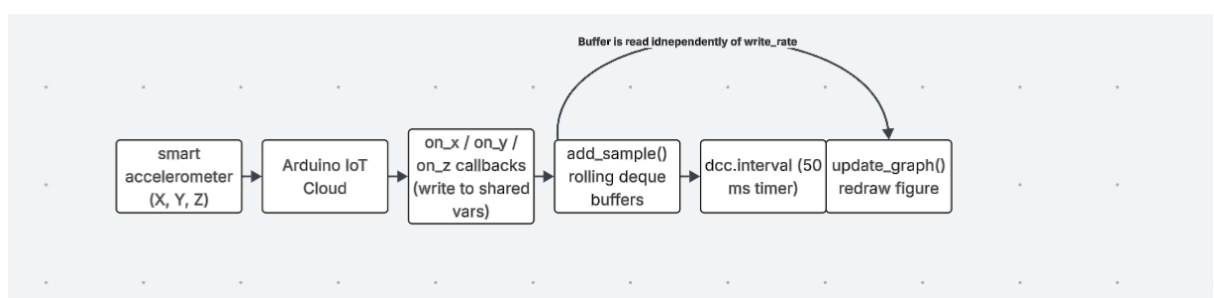
- `uirevision=True` on the layout keeps the zoom/pan state between each redraw so the view doesn't reset every time the figure object is replaced.
- `template="plotly_dark"` and a fixed margin/height keep the chart constant from frame-to-frame in size and style, avoiding layout “pop”.

Summary Table

Component	Role	Why it matters for smoothness
<code>on_x / on_y / on_z</code>	Arduino IoT Cloud write callbacks; fire independently whenever a new value for that single property arrives	Decouples network/cloud message arrival (irregular, bursty) from the graph redraw
<code>save_combined_data()</code>	Waits until X, Y and Z have all been received once, then writes one	Keeps the three channels time-

	synchronised row to CSV and calls add_sample()	aligned instead of drifting apart
add_sample() + deque(maxlen=200)	Rolling, fixed-length buffers for time, x, y, z	Bounded memory; oldest points fall off automatically so the plot always shows the latest window
threading.Thread(daemon=True)	Runs client.start() (blocking Arduino Cloud event loop) on a background thread	Keeps the Dash web server (app.run) responsive on the main thread; neither loop blocks the other
dcc.Interval(interval=50)	Fires update_graph() every 50 ms regardless of when cloud data actually arrived	Gives a constant, predictable redraw cadence — the visual frame rate is no longer tied to sensor jitter or network delay
uirevision=True	Told to Plotly to preserve UI state (zoom/pan) across figure replacements	Without it every redraw resets the view, which reads as a visible “jump” to the user

Flow diagram



II) Reusable Wrapper Function

The original `add_sample(x, y, z)` is hard-coded to exactly three named channels, so a peer working with, say, a single temperature sensor, or five channels of strain-gauge data, couldn't reuse it as-is. This has been abstracted into a small class, `StreamBuffer`, that accepts any number of named channels and exposes the same “append a sample, read back the buffer” pattern for any `dcc.Interval` driven Dash graph

API

"""

`StreamBuffer` — a generic rolling buffer for smooth Plotly Dash live updates.

This is the Q2 wrapper function/class referenced in the report. It is a generalised version of the accelerometer-only `add_sample(x, y, z)` used in `smooth_dash.py`: instead of being hard-coded to three channels, it accepts any number of named channels, so it can be reused for any live-streamed data (a single temperature sensor, five strain-gauge channels, etc.).

It is NOT wired into `smooth_dash.py` by default — that script still uses its own `add_sample(x, y, z)` to match the submitted demonstration video. This file exists purely as the documented, reusable API described in Q2.

"""

```
from collections import deque
from datetime import datetime
import threading
```

```
class StreamBuffer:
```

```
    """
```

```
    Generic rolling buffer for smooth Plotly Dash live updates.
```

```
    Wraps one or more named data channels (e.g. 'x', 'y', 'z', or
    'temperature', 'humidity', ...) in fixed-length deques, plus a
    matching timestamp deque, so any producer (cloud callback,
    serial read, sensor poll, ...) can push samples on arrival and
    any dcc.Interval callback can read the latest window without
    the two ever needing to run at the same rate.
```

```
    """
```

```

def __init__(self, channel_names, maxlen=200,
time_format="%H:%M:%S.%f"):
    self.channel_names = list(channel_names)
    self.maxlen = maxlen
    self.time_format = time_format
    self.time_buffer = deque(maxlen=maxlen)
    self.buffers = {name: deque(maxlen=maxlen) for name in
self.channel_names}
    self._lock = threading.Lock()

def add_sample(self, timestamp=None, **channel_values):
    """Append one synchronised sample across all given channels.

    timestamp : datetime or None. If None, uses datetime.now().
    channel_values : e.g. add_sample(x=1.2, y=-0.3, z=9.8)
    """
    t = (timestamp or datetime.now()).strftime(self.time_format)[:3]
    with self._lock:
        self.time_buffer.append(t)
        for name in self.channel_names:
            self.buffers[name].append(channel_values.get(name))

def get_trace(self, name):
    """Return (list_of_timestamps, list_of_values) for one channel,
    safe to call from a dcc.Interval callback at any time."""
    with self._lock:
        return list(self.time_buffer), list(self.buffers[name])

```

USAGE EXAMPLE: Accelerometer , 3 channels

```
buf = StreamBuffer(channel_names=["x", "y", "z"], maxlen=200)
```

inside the existing save_combined_data(), once x/y/z are all in:

```
buf.add_sample(x=x_value, y=y_value, z=z_value)
```

inside update_graph(_):

```
for axis in ("x", "y", "z"):
```

```
    t, values = buf.get_trace(axis)
```

```
    fig.add_trace(go.Scatter(x=t, y=values, mode="lines",
```

```
name=axis.upper()))
```

Usage example (single channel, e.g. temperature)

```
buf = StreamBuffer(channel_names=["temperature"], maxlen=300)
```

```
buf.add_sample(temperature=23.4)
```

```
t, values = buf.get_trace("temperature")
```

DESIGN JUSTIFICATION

- The channel_names are provided once when the class is constructed, so it does not make any assumptions regarding the number or the names of the axes – it can be used with either one channel or ten.
- The function add_sample(**channel_values) follows the same keyword format as the original callbacks (on_x, on_y, on_z), which means that switching to it will involve only minor modifications to the existing cloud-callback code.
- The threading.Lock is used to protect both deques since, in this project, the producer (the cloud callback thread) and the consumer (the Dash interval callback on the Flask thread) operate concurrently, and without the lock a redraw could read the buffer during the appending process.
- The function get_trace(name) returns plain Python lists rather than deque objects since Plotly's go.Scatter expects list-like x/y arguments and not deque views.
- The value of maxlen is provided as a parameter (rather than being fixed at 200) so that peers can balance their memory usage against the length of the window in relation to their own sample rate.

III) Demonstration Video

<https://youtu.be/aErYR56eoog>

The video demonstrates the script being run from start to finish: it establishes a connection with the Arduino IoT Cloud, receives the accelerometer data from the phone, and has the Dash graph updating smoothly at five0-millisecond intervals. The video was made using the original, task-specific version of the script (the one with the three-channel add_sample(x, y, z) function as shown in Q1). The generalised StreamBuffer wrapper that is described in Q2 is a drop-in replacement for that function which behaves in exactly the same way, it was designed and documented here to meet the Q2 requirement of providing a reusable API for peers, even though it is not shown running in the video.

IV) Repository

Link: https://github.com/srishti-9/SIT225_5C.git

